Tutorial Four: More Damped Oscillators

Robert is very excited, so he decides to pre-order a dessert to compose himself. Dan and I call this phenomenon?

1 RNN/GRU

In this part, we will train a Recurrent Neural Network (RNN) (one specific type called Gated Recurrent Unit (GRU)) to predict the future for the damped oscillators that we have worked with using PINN in the last tutorial.

a) In an RNN, the only input is the sequences of positions of the damped oscillator. How is this different from a standard NN or PINN that we used in tutorial 3?

   We will use the same technique as in tutorial 3 to generate the training data. In order to compare with the ground truth, we will only use 40% (or some other percentage) of the data for training.

   In the notebook, we have defined a very simple RNN.

b) Train with teacher forcing. Make sure that the error drops to negligibly small. Which line in the code implies that it is teacher forcing? (You may also want to look at the `forward` function in the RNN.)

c) Now we use free roll-out to predict the entire trajectory. Write a two line code to implement free roll-out: one line to retrieve the output and one line to use the previous output as the next step's input.

d) Check out the plot of the ground truth against the RNN prediction. How does RNN perform?

e) Write a code to plot the error as a function of time.

f) Compare your result to the standard NN that you have done in the last tutorial. This time we will feed NN more data to see if there is improvements.

2 Reservoir computing

The reservoir consists of a network of neurons, where the connections are fixed and are randomly initialized. Only the states h's of the reservoir evolve over time

$$h_{t+1} = \tanh(Wh_t + W_{in}x_t) \tag{1}$$

where x_t are the input, W are reservoir weight matrix and W_{in} are input projections. The output is then given by a simple linear mapping

$$y_t = W_{out}h_t \tag{2}$$

where y_t's are the target. W_{out} is trained via least-square regression.

$$W_{out} = \text{pseudo-inverse}(states) \cdot Y_{target} \tag{3}$$

where the states are all the hidden states h for all the training steps.

We start by defining a reservoir and initializing the connections.

a) In the definition of the reservoir, in which line are the states updated?

b) Train the reservoir with the same training data before. Compute the prediction on the hidden states generated during training and check the performance on the training data and examine the MSE.

   Now we are ready to make some real predictions. We will take the last training data as the input and the last column of states as the input h.

c) Write a code to update h, predict the next output *out* and generate the next input *inp*.

d) Run for a few steps, plot the free rollout prediction, and the ground truth. Comment on the training speed compared to NN's. Comment on stability.

e) One parameter we can play with to improve the stability is the size of the reservoir. What happens if the reservoir is too large? What happens if the reservoir is too small? Play around to find a reservoir that has at least some stable predictions.

3 Next-generation Reservoir Computing

It is called a reservoir computing, but there is no reservoir, at least in our simple example :P

One idea is that we can increase the number of features. For our example of the damped harmonic oscillator, in addition to the trajectory, we will also use the velocity data. (We can also infer the velocity from the trajectory, if we have time later, pun intended). For two features x and v, we can build three more quadratic features xx, xv, vv.

a) Write a code to produce these new features, complete the `poly_features(X)` function.

b) Compute Wout: this is very similar to RC before. Write a one line code to compute Wout.

c) Rollout prediction. Now for each data point, build the features for each timestep, and use the Wout computed in the last step to do the prediction.

d) See the plot of the prediction of NGRC.

e) Try to feed more scarce or noisy data to NGRC if have time.

4 Looking back and reflect

In the last two tutorials, we have applied an array of machine learning algorithms to a classical example, the damped harmonic oscillator. Here is a list of acronyms that we explored: NN, PINN, RNN, RC, NGRC.

a) Compare the training time and stability for all of them.

   It seems in this case we have a clear winner. The algorithm second in place seems to be an overkill in this case.

   Question continues in the next page (since you haven't seen the answer to the joke yet, it is obviously not the end of the tutorial). But I want you to rank them without seeing the answer.

b) So when do you think we need to use PINN over NGRC? Think about what the output of PINN is in addition to the predicted trajectory. Also, think of fluid dynamics.

c) When is NGRC particularly good? Think about something we should check every day if we are in Waterloo.

d) Any other thoughts?

Believe or not, the oscillator will return next week together with our good old friend Ising model.

Next-generation reserve, ah, calm pudding.